Management and Design of Databases (2231-1)

Instructions:

1. Create a student table. The student table needs to include the following columns:
    - Student ID
    - Student First Name
    - Student Last Name
    - Middle Name
    - City
    - State
    - ZIP Code
    - GPA
2. Use the appropriate data types for each column.
3. Create a Primary Key constraint for the table.
4. Make sure all columns contain either a NULL or NOT NULL constraint.

SQL Script:

```
USE PracticeDatabase
CREATE TABLE Student (
    StudentID int NOT NULL PRIMARY KEY,
    StudentFirstName varchar(255) NOT NULL,
    StudentLastName varchar(255) NOT NULL,
    MiddleName varchar(255), -- not all students will have a middle name
       City varchar(255) NOT NULL,
    State varchar(255) NOT NULL,
    ZipCode int NOT NULL,
    GPA decimal(3,2) NOT NULL,
       );
```

Instructions:

1. Using the student table you created in week two insert four rows into your table.
2. Update the first name to “Luke” and last name to “Plew” for one of the rows you inserted above.
3. Delete the row you updated above.
4. Explain how you insert data in one table from another table. Your answer should be a minimum of three sentences in length.

SQL Script:

```
VALUES ('480', 'John', 'Smith', 'Elmer', 'Albany', 'New York', '45687', '3.7')
INSERT INTO Student (StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA)
VALUES ('520', 'Jane', 'Smithson', 'Kim', 'Miami', 'Florida', '32785', '3.1')
INSERT INTO Student (StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA)
VALUES ('616', 'Samantha', 'Richards', 'June', 'Miami', 'Florida', '32784', '2.9')
INSERT INTO Student (StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA)
VALUES ('224', 'Bobby', 'Wallace', 'Richard', 'Orlando', 'Florida', '32189', '4.0')

UPDATE Student
SET StudentFirstName = 'Luke', StudentLastName = 'Plew'
WHERE StudentID = 520;

DELETE FROM Student WHERE StudentID ='520';

Select * from Student
```

Inserting Data from One Table to Another Explanation:

To insert data from one table to another you would use the INSERT INTO SELECT statement. In order for this to work correctly, the data types you are attempting to transfer must match the data types in the new table. You can either select all the contents of one table to add to the other table by using the SELECT * FROM along with the INSERT INTO SELECT statement. Or you can insert only some selected columns into the new table by specifying which specific columns you would like transferred.

Instructions:

Using the table you worked on in weeks 2 and 3, complete the following exercise:

1. Insert a row into your student table where the last name is "Parker", First name is "Sam" and the ZIP code is "68134". You can determine the values for the remaining columns.
2. Select a row from your student table where the last name equals "Parker".
3. Select a row from your student table where the ZIP code is less than "68134".
4. Select the student first and last name from the student table order by the state column.

SQL Script:

```
INSERT INTO Student(StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA)
VALUES ('513', 'Sam', 'Parker', 'Edwin', 'Omaha', 'Nebraska', '68134', '3.8')

SELECT * FROM Student
WHERE StudentLastName='parker';
```

```sql
SELECT * FROM Student
WHERE ZipCode < 68134;

SELECT StudentFirstName, StudentLastName FROM Student
ORDER BY State;
```

Instructions:

1. Using your student table select all students that have a last name of “Parker” and have a first name that starts with “S”. Order your results by state.
2. Using your student table select all students with ZIP codes between “23164” and “98164”.
3. Using your student table select all students where states are in Florida and New York.
4. Using your student table select all students where the last name starts with an “S” and is at least three characters in length.

SQL Script:

```sql
SELECT * FROM Student
WHERE StudentLastName LIKE '%Parker%' AND StudentFirstName LIKE 's%'
ORDER BY State;

SELECT * FROM Student
WHERE ZipCode BETWEEN 23164 AND 98164;

SELECT * FROM Student
WHERE State IN ('Florida', 'New York');

SELECT * FROM Student
WHERE StudentLastName LIKE 's__%';
```

Question 1: Using your student table, what is the average GPA?

The average GPA for my student table is 3.600000

SQL Script:

```sql
SELECT AVG(GPA) AS AverageGPA
FROM Student;
```

Question 2: Using your student table, what is the minimum GPA?

The minimum GPA for my student table is 2.90

SQL Script:

```
SELECT MIN(GPA) AS MinimumGPA
FROM Student;
```

Question 3: Explain why the following select statement is incorrect (your answer should be limited to no more than a few sentences):

- SELECT COUNT(first_name) FROM student

The select statement is incorrect because the table heading "first_name" is not one of the names of the columns in the table. The correct column name is StudentFirstName. Replacing a correct column name and running the query will give you a result of 4 because there are 4 rows in the table that contain a first name (any null values would be ignored).

Question 4: Explain the following select statement (your answer should be limited to no more than a few sentences):

- SELECT MIN(DISTINCT gpa) FROM student

The DISTINCT statement does not include any duplicate values. When the DISTINCT is placed within the parentheses, it is carried out first... The distinct values are identified first, then the minimum is calculated from those distinct values.

SQL Script:

1. Using your student table what is the average GPA for each state?

```
SELECT AVG(GPA) AS AVG_GPA, State
FROM Student
GROUP BY State
ORDER BY AVG(GPA)
```

2. Using your student table what is the average GPA where students are in Florida or New York?

```
SELECT AVG(GPA) AS AVG_GPA, State
FROM Student
GROUP BY State
HAVING State = 'Florida' OR State = 'New York'
```

3. Using your student table return the average GPA for each state having an average GPA greater than 3.

```
SELECT AVG(GPA) AS AVG_GPA, State
FROM Student
GROUP BY State
```

```
HAVING AVG(GPA) > 3
```

4. Using your student table how many students from each state have an average GPA less than 2?

```
SELECT State, COUNT(GPA) AS NUM_OF_STUDENTS
FROM Student
WHERE GPA < 2
GROUP BY State
```

1. Using your student table return all students with their first and last name concatenated. Make sure there is a space between the first and last name.

SQL Script:

```
SELECT StudentFirstName + ' ' + StudentLastName AS StudentFullName
FROM Student
```

2. Using your student table return the distinct students last name in all upper case.

SQL Script:

```
SELECT UPPER(StudentLastName) AS UpperLastName
FROM Student
```

3. Using your student table return the first three character of the students last name.

SQL Script:

```
SELECT SUBSTRING(StudentLastName, 1, 3) AS FirstThreeLetters
FROM Student
```

4. Using your student table replace all the “A” characters with the letter “J”.

SQL Script:

```
SELECT StudentID,
REPLACE(StudentFirstName, 'A', 'J') AS StudentFirstName,
REPLACE(StudentLastName, 'A', 'J') AS StudentLastName,
REPLACE(MiddleName, 'A', 'J') AS MiddleName,
REPLACE(City, 'A', 'J') AS City,
REPLACE(State, 'A', 'J') AS State,
ZipCode,
GPA
FROM Student
```

1. Alter your student table adding a new column called enroll_date using the datetime data type.

```
ALTER TABLE Student
ADD Enroll_Date DATETIME
```

2. Populate the new column (enroll_date) with the current date and time.

```
UPDATE Student
SET Enroll_Date = GETDATE()
```

3. Using your student table return the student first and last names concatenated with a space and then the enroll_date minus one month. Here is an example of the output:

Name	Date
Jane Smith	2019-05-03 13:14:12
Tom Jones	2020-08-07 05:23:56

```
SELECT StudentFirstName + ' ' + StudentLastName AS Name, Enroll_Date = DATEADD(MONTH,-1, '2022-10-24 10:40:08.067')
FROM Student
```

4. Using your student table return the month name for the enroll_date column. For example, if the enroll_date was 11-6-2019 then the month name would be June.

```
SELECT Enroll_Date = DATENAME(MONTH, '2022-10-24 10:40:08.067')
FROM STU_Table
```

1. Create the following table with the correct columns. Call your new table: student_course (see below). Populate this table with four rows. Make sure the student id column contains values that are also in the student id column of your student table.
   a. student id
   b. course id
   c. semester
   d. year

<u>SQL Script</u>:

```
CREATE TABLE Student_Course (
  StudentID int NOT NULL PRIMARY KEY,
  CourseID int NOT NULL,
```

```
 Semester varchar(255) NOT NULL,
 Year varchar(255) NOT NULL,
);

INSERT INTO Student_Course(StudentID, CourseID, Semester, Year)
VALUES ('224', '2800', 'Fall', '2020')
INSERT INTO Student_Course(StudentID, CourseID, Semester, Year)
VALUES ('480', '2600', 'Winter', '2022')
INSERT INTO Student_Course(StudentID, CourseID, Semester, Year)
VALUES ('513', '2800', 'Spring', '2023')
INSERT INTO Student_Course(StudentID, CourseID, Semester, Year)
VALUES ('616', '3000', 'Fall', '2020')
```

2. Join the student and student_course table where the student id values are the same.

SQL Script:

```
SELECT Student.StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA, CourseID, Semester, Year
FROM Student_Course
INNER JOIN Student ON Student_Course.StudentID=Student.StudentID;
```

3. Join the student and student_course table where the student id values are the same and year is equal to 2020.

SQL Script:

```
SELECT Student.StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA, CourseID, Semester, Year
FROM Student_Course
INNER JOIN Student ON Student_Course.StudentID=Student.StudentID
WHERE student_course.year = '2020'
```

4. Create a select statement that will return the length of the last name column by joining the student and student_course table where the student id values are the same and enroll month is June.

**** Due to a typo in this question the query was made using the following edits:** "Create a select statement that will return the length of the last name column by joining the student and student_course table where the student id values are the same and **semester** is **Fall**."

SQL Script:

```
SELECT LEN(StudentLastName) AS LengthLNameColumn
FROM Student_Course
INNER JOIN Student ON Student_Course.StudentID=Student.StudentID
WHERE Semester = 'fall'
```

1. Using a subquery, select student first name where the student is in the student course table.

SQL Script:

```
SELECT StudentFirstName
FROM Student
WHERE StudentID IN (SELECT StudentID FROM Student_Course)
```

2. Using a subquery, select the course id where the average GPA is 2.

SQL Script:

```
SELECT CourseID
FROM Student_Course, Student
WHERE GPA = (SELECT AVG(GPA) FROM Student
WHERE Student_Course.StudentID = Student.StudentID AND Student.GPA = 2)
GROUP BY CourseID
```

3. Using a subquery, select the course id where the average GPA is greater than 2.

SQL Script:

```
SELECT CourseID
FROM Student_Course, Student
WHERE GPA = (SELECT AVG(GPA) FROM Student
WHERE Student_Course.StudentID = Student.StudentID AND Student.GPA > 2)
GROUP BY CourseID
```

4. Using a subquery, select student first name where the student is not in the student course table.

*All students that were in the student table were also in the student course table... Therefore, a new student was first added to the student table so that this query could be tested for correctness...

```
INSERT INTO Student(StudentID, StudentFirstName, StudentLastName, MiddleName, City, State, ZipCode, GPA, Enroll_Date)
VALUES ('355', 'May', 'Jones', 'Elizabeth', 'LA', 'California', '56789', '2', GETDATE())
```

SQL Script:

```
SELECT StudentFirstName
FROM Student_Course, Student
WHERE Student.StudentID = ANY (SELECT StudentID FROM Student
WHERE Student_Course.StudentID = Student.StudentID)
```

1. Using a union, select the second number from the student id column from both the student and student course tables. For example, if the student ID was “123456” then you would select “2”. You should return the second number for all rows in these two tables.

   SQL Script:

```
SELECT SUBSTRING(StudentID, 2, 1) FROM Student
UNION
SELECT SUBSTRING(StudentID, 2, 1) FROM Student_Course
```

2. Using a union, select all the student ids from both the student and student course tables replacing any occurrence of “1” with “0” in your output.

   SQL Script:

```
SELECT
REPLACE(StudentID, '1', '0') AS StudentID
FROM Student
UNION
SELECT
REPLACE(StudentID, '1', '0') AS StudentID
FROM Student_Course
```

3. Using a union, select all the student ids from both the student and student course tables displaying the first three numbers of the student id followed by a dash then the complete student id. For example, if the student Id is 134567 then the result would be 134-134567.

   SQL Script:

```
SELECT SUBSTRING(StudentID, 1, 3) + '-' + StudentID AS StudentIDLong FROM Student
UNION
SELECT SUBSTRING(StudentID, 1, 3) + '-' + StudentID AS StudentIDLong FROM Student_Course
```

4. Using a union, select only the first number of the student id and the count of that number in the output from the student and student course tables. Hint you will have to use a group by with your union statement. Here is an example of the output:

   SQL Script:

```
SELECT SUBSTRING(StudentID, 1, 1) AS StudentID, COUNT(SUBSTRING(StudentID, 1, 1)) AS Count
FROM Student4
GROUP BY StudentID
UNION
SELECT SUBSTRING(StudentID, 1, 1) AS StudentID,
COUNT(SUBSTRING(StudentID, 1, 1)) AS Count
FROM Student_Course4
GROUP BY StudentID
```

Student_ID	Count
1	6
2	3
3	1
4	4